

Task 1:

Write some code that takes a list of numbers, which we'll call `nums_list` (very creative, I know), and for each number, prints out:

- The square root of the number,
- The number itself,
- The number, squared,
- The number, cubed

All separated by commas.

For instance if our list contains `[1, 2]`, the your code should print:

1.0, 1, 1, 1

1.41421356237, 2, 4, 8

Hint – you'll want to use the exponentiation operator -- `**`. `2**4` is 24, and you can use `**0.5` to take a square root.

You can initialise your list with something like the following:

```
nums_list = [1, 2, 3, 4, 5]
```

We'll talk more about lists in a couple weeks.

Task 2:

Use a `while` loop to prompt the user to enter in a value between 1 and 10, as many times as it takes for them to get it right.

Task 3:

Prime numbers are ones divisible only by 1 and themselves. Write some code that will determine, and print, if a given integer number is prime or not.

Hint: The remainder operator (`%`) is going to be quite handy.

Task 4:

Computers approximate complicated functions from simpler ones. Write a program that calculates the n^{th} order Taylor series approximation of $\cos(x)$ where n and x come from the user, according to the following formula:

$$\cos(x) = \sum_{k=0}^n (-1)^k \frac{x^{2k}}{(2k)!}$$

The program should also print the percent error of the approximation as compared to Python's `math.cos` function.

$$per_err = 100 * \frac{|y_{actual} - y_{estimated}|}{y_{actual}}$$

Hint: Calculate $(2k)!$ using `math.factorial(2*k)`

Example run:

```
Enter x: 1.5
Enter n: 4
cos(x) = 0.07075282505580358
Percent error = 0.022086522695740584
```

What happens when you increase n ?